

OBJECT ORIENTED PROGRAMMING

S.Gayathri

Assistant Professor

Department of Computer Science and Engineering

Jerusalem College of Engineering

UNIT-III: FILES, TEMPLATES AND EXCEPTION HANDLING

Files: File Stream Operations - File Pointers Manipulation-
Templates: Function templates – Class Templates – Exception
handling: Try, Catch and Throw- Handling Derived-Class
Exceptions - Exception Handling Options.

FILES

FILES

- A file is a collection of related data stored in a particular area on the disk.
- A program typically involves either a both of the following kinds of data communication.
 - 1.data transfer between console(keyboard) unit and program
 - 2.data transfer between program and disk file.
- In c++ file uses stream as an interface between the programs and files.
- The stream that supplies data to the program is known as **input stream** (reads data from file).
- The stream that receives data from the program is known as **output stream** (write data to the file).

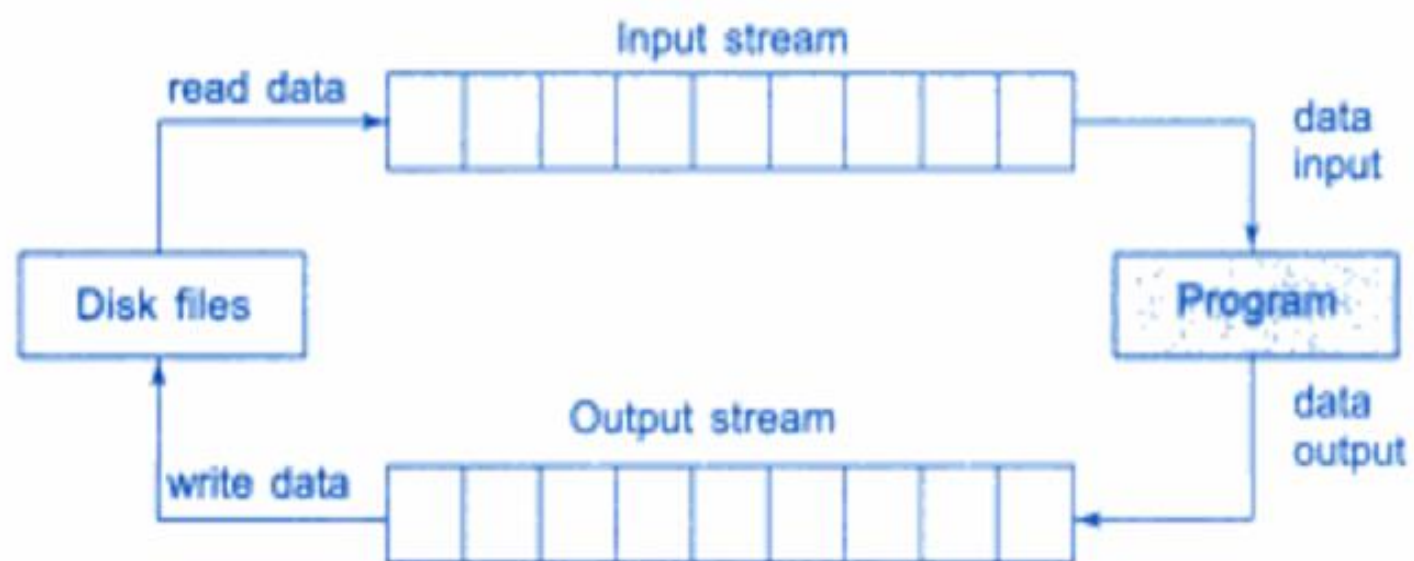


Fig. 11.2 $\Leftrightarrow$ File input and output streams

File stream classes

The different file stream classes are

- **filebuf** – sets the file buffer to read and write, has open() and close() member functions.
- **filestreambase** – serves as a base class to fstream, ifstream and ofstream classes, contains open() and close() functions.
- **ifstream** – provides input operation, contains open() with i/p mode. Inherits some function get(),getline(),read(),seekg(),and tellg functions from istream
- **ofstream** – provides o/p operation, contains open() with default o/p mode inherits put(),seekp(), tellp() and write() from ostream.
- **fstream** – i/o operation inherits from istream and ostream through iostream.

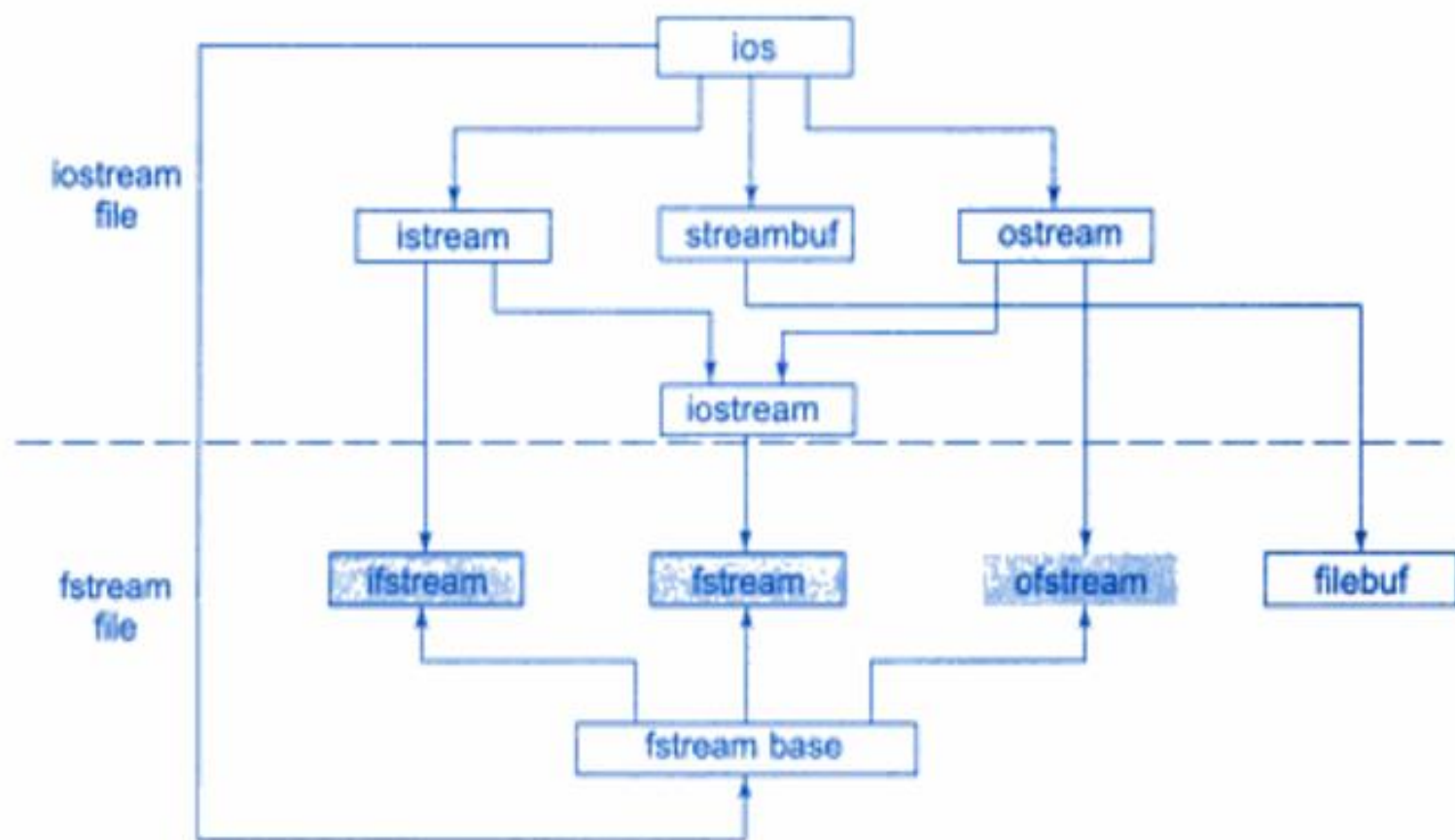


Fig. 11.3 ⇔ Stream classes for file operations (contained in *fstream* file)

Opening and closing of file

- When a program is written using files it must have an header file **<fstream.h>**
- A file can be opened in **two ways**
 1. using constructor function of the class
 2. Using member function open() of the class

1.opening a file using a constructor

this involves 2 steps

- (a) create a file stream object of an appropriate class ofstream to create an o/p stream and ifstream to create an i/p stream.
- (b) initialize the file object with desired filename.

For example the following statement open's a file named "result " for output

ofstream outfile("result")

outfile is an object of the class ofstream

similarly

ifstream infile("data")

infile is an object of the class ifstream and file named "data" is opened for reading

Example:

```
#include<fstream.h>
main()
{
ofstream outfile ("item");
char name[30];
cin>>name; //reads name from the user
outfile<<name; //puts into the file item
outfile.close;
ifstream infile("item")
infile>>name; // reads from the file
cout<< "name"<<name; // prints on the screen
Infile.close();
}
```

2.opening files using open()

open() can be used to open many files when it is required. This can be done as follows.,

Filestream class stream object;

Stream object. Open("filename");

Example:

Ofstream outfile;

Outfile.open("data1");

outfile.close();

outfile.open("data2")

here is the above example first data1.file is opened after manipulating with that file it is closed . then again another file is opened ie. data2.

// creating files with open() function

```
#include <iostream.h>
```

```
#include <fstream.h>
```

```
int main()
```

```
{
```

```
    ofstream fout; // create output stream
```

```
    fout.open("country");
```

```
    fout << "United States of America\n";
```

```
    fout << "United Kingdom\n";
```

```
    fout << "South Korea\n";
```

```
    fout.close();
```

```
    fout.open("capital");
```

```
    fout << "Washington\n";
```

```
    fout << "London\n";
```

```
    fout << "Seoul\n";
```

```
    fout.close();
```

// Reading the files

```
const int N = 80;
char line[N];
ifstream fin;
fin.open("country");
cout << "contents of country file\n";
while(fin)
{
    fin.getline(line, N);
    cout << line;
}
fin.close();
```

```
fin.open("capital");
cout << "\nContents of capital
file\n";
while(fin)
{
    fin.getline(line, N);
    cout << line;
}
fin.close();
return 0;
}
```

Output:

Contents of country file

United States of America

United Kingdom

South Korea

Contents of capital file

Washington

London

Seoul

Detecting end of file

- Detection of EOF is necessary for preventing any further attempt to read data from the file.

While(fin)

- **fin** returns a value 0 when it reaches end of file. And also returns 0 when any other error occurs so they started using another statement.

```
if(file.eof() != 0)
```

```
{
```

```
    exit(1);
```

```
}
```

to check the EOF.

File open modes

General syntax for opening a file using the modes is

Streamobject.open("filename", mode);

Default values are as follows

ios::in → open for read only

ios::out → open for writing only

Table 11.2 *File mode parameters*

<i>Parameter</i>	<i>Meaning</i>
ios :: app	Append to end-of-file
ios :: ate	Go to end-of-file on opening
ios :: binary	Binary file
ios :: in	Open file for reading only
ios :: nocreate	Open fails if the file does not exist
ios :: noreplace	Open fails if the file already exists
ios :: out	Open file for writing only
ios :: trunc	Delete the contents of the file if it exists

File Pointers and their Manipulations

- Each File has two associated pointers known as the file pointers.

(i). input pointer(get pointer)

(ii).output pointer(or put pointer)

- We can use these pointers to move through the files while reading or writing.
- The input pointer is used for reading the contents of the given file location and the output pointer is used for writing to a given file location.
- Each time an output operation takes place, the appropriate pointer is automatically advanced.

DEFAULT ACTIONS:

- When we open a file in read –only mode ,the input pointer is automatically set at the beginning so that we can read the file from the start.
- Similarly, when we open a file in write-only mode, the existing contents are deleted and the output pointer is set at the beginning.
- This enables us to write the file from the start. In case, we want to open an existing file to add more data, the file is opened in ‘append’ mode. This moves the output pointer to the end of the file(i.e. the end of the existing contents).

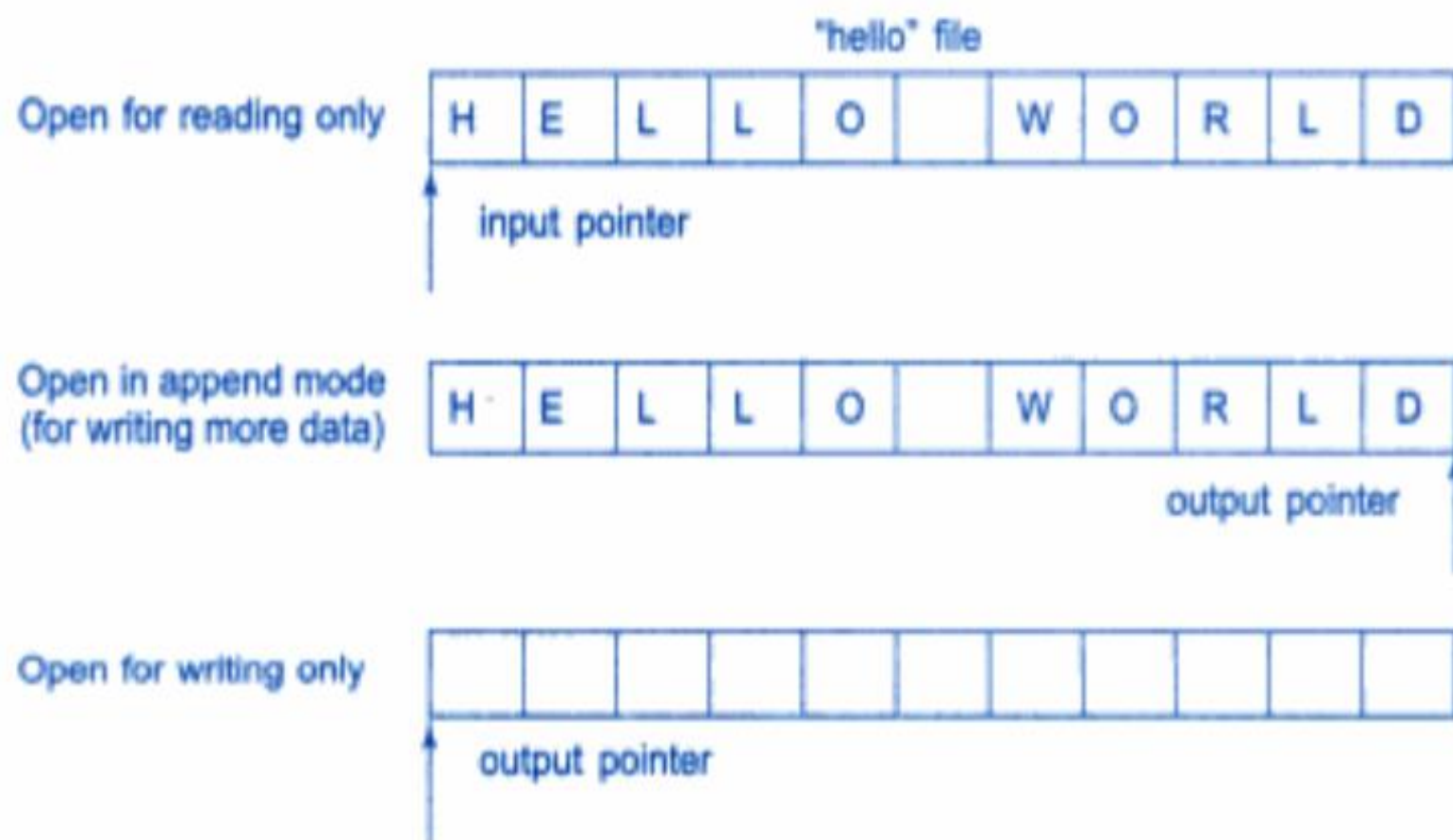


Fig. 11.7 ⇔ Action on file pointers while opening a file

FUNCTIONS FOR MANIPULATION OF FILE POINTERS

- All the actions on the file pointers take place automatically by default. How do we then move a file pointer to any other desired position inside the file? This is possible only if we can take control of the movement of the file pointers ourselves.
- The file stream classes support the following functions to manage such situations:
 - `seekg()` Moves get pointer (input) to a specified location.
 - `seekp()` Moves put pointer(output) to a specified location.
 - `tellg()` Gives the current position of the get pointer.
 - `tellp()` Gives the current position of the put pointer.

Example

`infile.seekg(10);` → moves the file pointer to the byte no 10.

Consider the following statements:

`ofstream fileout;`

`fileout.open("hello",ios::app);`

`int p= fileout.tellp();` → the value of `p` will represent the number of bytes in the file.

Specifying the offset

How to move a file pointer to a desired location using the 'seek' functions.

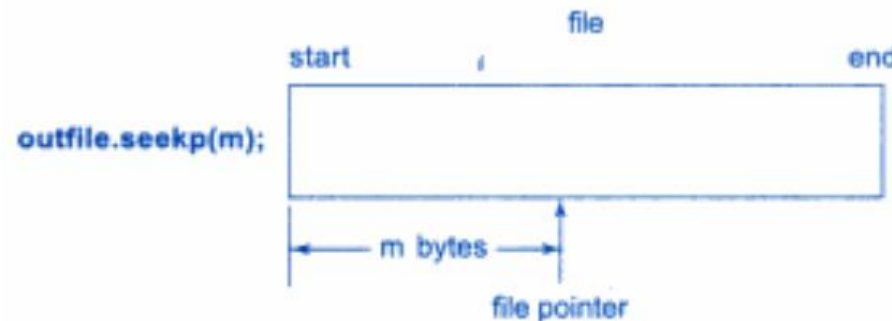


Fig. 11.8 ⇔ Action of single argument seek function

- Seekg() and seekp() can be used with two arguments as follows:

```
seekg (offset, reposition);  
seekp (offset, reposition);
```

- The parameter offset represents the number of bytes the file pointer is to be moved from the location specified by the parameter reposition.
- The reposition takes one of the following three constants defined in the ios class:
 - (i).ios::beg → start of the file
 - (ii).ios::cur → current position of the pointer
 - (iii).ios::end → end of the file.

Table 11.3 *Pointer offset calls*

<i>Seek call</i>	<i>Action</i>
<code>fout.seekg(0, ios::beg);</code>	Go to start
<code>fout.seekg(0, ios::cur);</code>	Stay at the current position
<code>fout.seekg(0, ios::end);</code>	Go to the end of file
<code>Fout.seekg(m, ios::beg);</code>	Move to (m + 1)th byte in the file
<code>fout.seekg(m, ios::cur);</code>	Go forward by m byte form the current position
<code>fout.seekg(-m, ios::cur);</code>	Go backward by m bytes from the current position
<code>fout.seekg(-m, ios::end);</code>	Go backward by m bytes form the end

Exception handling

- The two most common types of bugs are logic errors and syntactic errors.
- The **logic errors** occur due to poor understanding of the problem and solution procedure.
- The **syntactic errors** arise due to poor understanding of the language itself.
- We often come across peculiar problems other than logic or syntax errors. They are known as exceptions.
- Exceptions are run time anomalies or unusual conditions that a program may encounter while executing.
- **Eg:** divisions by zero, access to an array outside of its bounds, or running out of memory or disk space.

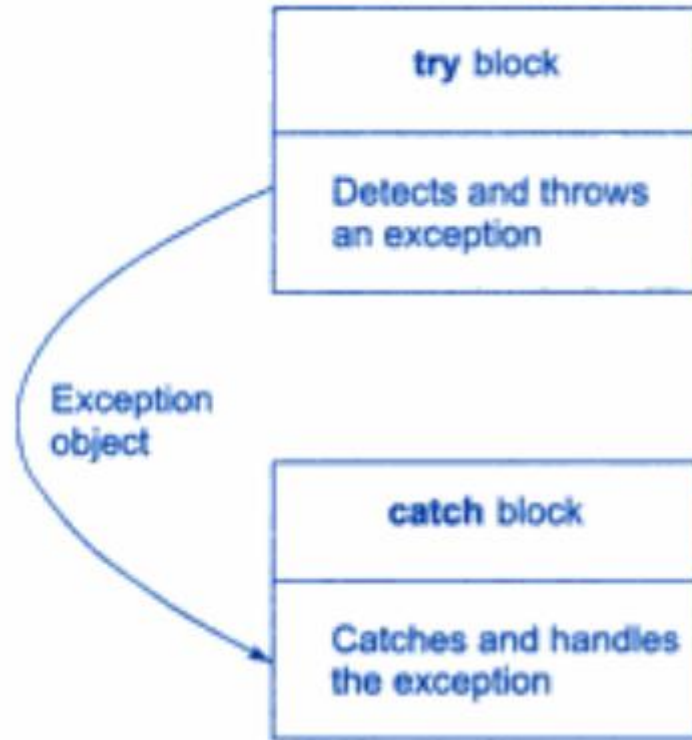
Basics of Exception Handling

- The purpose of exception handling mechanism is to provide means to detect and report an exceptional circumstance so that an appropriate action can be taken.
- The mechanism suggests a separate error handling code that performs the following tasks.
 1. Find the problem (Hit the exception)
 2. Inform that an error has occurred (Throw the exception)
 3. Receive the error information (Catch the exception)
 4. Take corrective actions (Handle the exception)

Exception Handling Mechanism

- It is built upon three keywords, namely **try**, **throw** and **catch**.
- The keyword **try** is used to preface a block of statements which may generate exceptions. This block of statements is known as try block.
- When an exception is detected it is thrown using a **throw** statement in the try block.
- A catch block define by the keyword catch 'catches' the exception thrown by the throw statement in the try block and handles it appropriately.

The block throwing exception



General Form

....

....

try

{

 throw exception ; // block of statements which detect and throw an exception

 ...

 ...

}

catch(type arg) // catches exception

{

 ... // block of statements which handles the exception

 ...

 ...

 ...

}

Eg:

```
int main()
{
    int a,b;
    cout << "Enter the values of a and b\n";
    cin >> a >> b;
    int x = a-b;
    try
    {
        if(x != 0)
        {
            cout << "Result(a/x) = " << a/x << "\n";
        }
        else    // There is exception
        {
            throw(x); // Throws int object
        }
    }
}
```

```
catch(int i) // Catches the exception
{
    cout << "Exception caught: x = " << x
    << "\n";
}

    cout << "END";

    return 0;
}
```

Output:

Enter values of a and b

20 15

Results(a/x) = 4

END

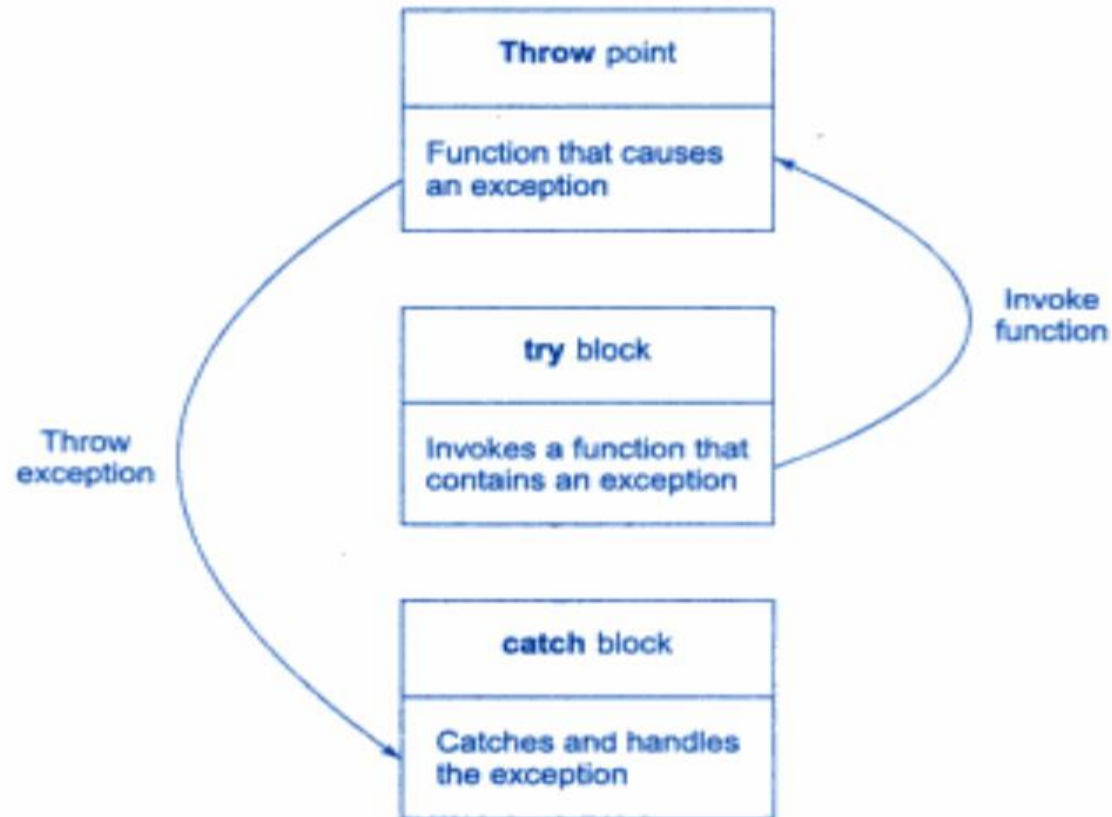
Enter values of a and b

10 10

Exception caught: $x = 0$

END

- Exceptions are thrown by functions that are invoked from within the try blocks.
- The point at which the throw is executed is called the throw point. once an exception is thrown to the catch block, control cannot return to the throw point.



General Form

```
type function(arg list)  // function with exception
{
    ....
    ...
    throw(object);  // Throws exception
    ...
    ...
}
...
...
try
{
    ...
    ... // Invoke function here
    ...
}
```

```
catch(type arg)
{
    ...
    ... // Handles exceptions here
    ...
}
...
...
```

Invoking function that generates Exception:

```
void divide(int x, int y, int z)
{
    cout << "\nWe are inside the function\n";
    if((x-y) != 0) // It is ok
    {
        int R = z/(x-y);
        cout << "Result = " << R << "\n";
    }
    else // There is a problem
    {
        throw(x-y); // Throw point
    }
}
```

```
int main()
{
    try
    {
        cout << "We are inside the try block \n";
        divide(10,20,30); // invoke divide()
        divide(10,10,20); // invoke divide()
    }
    catch(int i) // Catches the exception
    {
        cout << "Caught the exception \n";
    }
    return 0;
}
```

Output:

We are inside the try block
We are inside the function
Result=-3
We are inside the function
Caught the exception

Types of Exception

- 1.Division by zero
- 2.Null pointer
- 3.Stack overflow
- 4.Array index out of bound

Throwing Mechanism

- When an exception that is desired to be handled is detected, it is thrown using throw statement in one of the following
 - `throw(exception);`
 - `throw exception;`
 - `throw; //used for rethrowing an exception`
- When exception occurred, the control exit the current try block and is transferred to the catch block.

Catching mechanism

- code for handling exceptions is included in catch blocks.
- Catch(type arg)
- {
- //stmt for managing exceptions
- }

• **Multiple Catch**

- It is possible that a program segment has more than one condition to throw an exception. we can have more than one catch statement with a try

Multiple Catch Statements:

```
try
{
    // try block
}
catch(type1 arg)
{
    // catch block1)
}
catch(type2 arg)
{
    // catch block2
}
...
...
catch(typeN arg)
{
    // catch blockN
}
```

MULTIPLE CATCH STATEMENTS

```
void test(int i)
{
    try
    {
        if(x == 1) throw x; // int
        else if (x == 0) throw 'x'; // char
        else if (x == -1) throw 1.0; // double
        cout << "end of try block";
    }
    catch(char c) // catch 1
    {
        cout << "caught a character\n";
    }
    catch(int m) // catch 2
    {
        cout << "caught an integer\n";
    }
    Catch(double d) // catch 3
    {
        cout << "caught a double\n";
    }
    cout << "end of try catch\n";
}
```

```
int main()
{
    cout << "Testing multiple catches\n";
    cout << "x == 1\n";
    test(1);
    cout << "x == 0\n";
    test(0);
    cout << "x == -1\n";
    test(-1);
    cout << "x == 2\n";
    test(2);
    return 0;
}
```

Testing multiple catches

X == 1

Caught an integer

End of try catch

X == 0

Caught a character

End of try catch

X == -1

Caught a double

End of try catch

X == 2

End of try block

End of try catch

Template

- Template provide support to Generic programming.
- Generic programming is an approach where generic types are used as parameters in algorithms so that they work for a variety of suitable data types and data structures.
- A template can be considered as a kind of macro. When an object of a specific type is defined for actual use, the template definition for that class is substituted with the required data type. Since a template is defined with a parameter that would be replaced by a specific data type at the time of actual use of the class or function, the templates are sometimes called parameterized classes or functions.

Function Template

```
#include <iostream.h>
template<class T> void swap(T *a, T*b)
{
    T temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

```
void main(void)
{
    int a = 8, b = 12;
    float c = 29.5, d = 31.8;
    double e = 8.9, f = 51.7;
    cout << "before swap" << endl;
    cout << "a=" << a << " b=" << b << endl;
    cout << "c=" << c << " d=" << d << endl;
    cout << "e=" << e << " f=" << f << endl;
    swap(&a,&b);
    swap(&c,&d);
    swap(&e,&f);
    cout << "after swap" << endl;
    cout << "a=" << a << " b=" << b << endl;
    cout << "c=" << c << " d=" << d << endl;
    cout << "e=" << e << " f=" << f << endl;
}
```

Output:

before swap

a=8 b=12

c=29.5 d=31.8

e=8.9 f=51.7

after swap

a=12 b=8

c=31.8 d=29.5

e=51.7 f=8.9

Class Template

```
#include <iostream.h>
template<class T> class Greatest
{
    private:
        T num1, num2, num3;
    public:
        Greatest(T number1, T number2, T number3)
        {
            num1 = number1;
            num2 = number2;
            num3 = number3;
        }
}
```

```
T FindGreatest()
```

```
{
```

```
    if(num1 > num2 && num1 > num3)
```

```
    {
```

```
        return num1;
```

```
    }
```

```
    else if(num2 > num1 && num2 > num3)
```

```
    {
```

```
        return num2;
```

```
    }
```

```
    return num3;
```

```
}
```

```
};
```

```
void main(void)
{
    Greatest<int> g(17, 9, 51);
    Greatest<float> f(71.2, 911.147, 811.218);
    Greatest<double> d(121.417, 79518.59134, 21192.127);
    clrscr();
    cout << "Greatest int      is " << g.FindGreatest() << endl;
    cout << "Greatest float is " << f.FindGreatest() << endl;
    cout << "Greatest double is " << d.FindGreatest() << endl;
    getch();
}
```

Output:

Greatest int is 51

Greatest float is 911.146973

Greatest double is 79518.59134